ÇANKAYA UNIVERSITY
COMPUTER ENGINEERING DEPARTMENT

CENG 408

Experiments and Results

AI / ML Modules: Disease Detection and LLM-Based Advisory

TARAS: Agricultural Analysis System

ARDA KAHRAMAN - 202111002
HATİCE ELİF ARABACI - 202211008
ÖYKÜ EYLÜL BOZDAĞ - 202311410
MUSTAFA BERK KESKİN - 202211042

1. Introduction..... 3
1.1 Purpose of This Document..... 3
2. Disease Detection Module..... 3
2.1 Evaluation Philosophy..... 3
2.2. Experimental Design Principles..... 4
2.2.1 Real-World-Oriented Evaluation Strategy..... 4
2.2.2 Fair Model Comparison Protocol..... 4
2.2.3 Separation of Benchmark vs Deployment Experiments..... 5
2.3. Dataset and Data Pipeline..... 6
2.3.1 Dataset Sources..... 6
2.3.2 Dataset Construction Strategy..... 6
2.3.3 Data Cleaning and Validation..... 7
2.3.4 Data Preprocessing Pipeline..... 8
2.3.5 Data Augmentation Strategy..... 8
2.3.6 Dataset Splitting Strategy..... 9
2.4. Model Architectures..... 10
2.4.1 CNN-Based Models..... 10
2.4.2 Transformer-Based Models..... 10
2.4.3 Architectural Differences..... 11
2.5. Training Configuration..... 12
2.5.1 Optimization Strategy..... 12
2.5.2 Sampling Strategy..... 13
2.5.3 Hyperparameters..... 13
2.6. Evaluation Metrics..... 14
2.6.1 Classification Metrics..... 14
2.6.2 Metric Justification..... 15
2.6.3 Validation Selection Metric..... 15
2.7. Experimental Results..... 16
2.7.1 Benchmark Comparison Results..... 16
2.7.2 Final Model Results..... 17
2.7.3 Per-Class Performance Analysis..... 18
2.7.4 Confusion Matrix Analysis..... 20
2.8. Generalization and Robustness Analysis..... 22
2.8.1 Real-World Generalization..... 22
2.8.2 Class Stability Analysis..... 22
2.8.3 Failure Case Analysis..... 23
3. LLM Module Evaluation..... 24

1. Introduction

1.1 Purpose of This Document

This document presents the experimental evaluation of the machine learning components in the TARAS platform. It reports the methodology, evidence, and observed performance of the models under both controlled and deployment-oriented conditions.

The evaluation covers two subsystems: the vision-based disease detection module and the LLM-based advisory module. Since these address different tasks, they are evaluated differently: the disease module with quantitative classification metrics, and the advisory module with qualitative, scenario-based testing.

2. Disease Detection Module

2.1 Evaluation Philosophy

The disease detection evaluation focuses on real-world generalization rather than laboratory-level accuracy. Although models trained on controlled datasets such as PlantVillage can achieve very high scores, these results may not transfer well to field conditions because of differences in lighting, background complexity, and image variability. Therefore, TARAS uses real-world performance as the main indicator of model quality.

For this reason, natural-condition datasets are prioritized in validation and testing. The real-world test set (test_pd) is treated as the primary benchmark for deployment readiness, while additional datasets are used to support analysis and check consistency. This ensures that the reported results reflect practical use rather than idealized laboratory conditions.

Overall, the evaluation emphasizes robustness, consistency, and practical applicability, so the selected model is not only accurate but also reliable in real agricultural environments.

2.2. Experimental Design Principles

2.2.1 Real-World-Oriented Evaluation Strategy

The evaluation strategy reflects real deployment conditions, where agricultural images are captured with varying lighting, complex

backgrounds, and partial occlusions. For this reason, laboratory-style datasets alone are not sufficient to measure real-world performance.

The real-world test_pd dataset is used as the primary benchmark, and deployment decisions are based mainly on its results. The test_balanced dataset is used as a secondary benchmark to examine class-level behavior under a more controlled distribution.

This setup allows TARAS to evaluate both practical field performance and class-level consistency, reducing the risk of overestimating model quality under ideal laboratory conditions.

2.2.2 Fair Model Comparison Protocol

All candidate architectures are evaluated under the same experimental setup to ensure a fair comparison. Each model uses the same dataset configuration, preprocessing pipeline, augmentation strategy, hyperparameters, optimization method, train/validation splits, test sets, and evaluation metrics.

This controlled setup ensures that performance differences are caused mainly by model architecture rather than inconsistent training conditions. As a result, model selection is based on reproducible experimental evidence instead of subjective preference or uncontrolled variables.

2.2.3 Separation of Benchmark vs Deployment Experiments

The evaluation separates benchmark comparison from final model selection, but both use the same dataset, splits, metrics, and evaluation framework. In the benchmark stage, multiple architectures are trained and tested under identical conditions to compare their performance fairly.

The final deployment model is then selected from these results. Since all candidates are evaluated on the same revised dataset, the model with the strongest overall performance becomes the deployment candidate.

This keeps model selection transparent and evidence-based, relying on direct experimental comparison rather than subjective preference or architectural assumptions.

2.3. Dataset and Data Pipeline

2.3.1 Dataset Sources

The dataset combines laboratory and real-world image sources to address domain shift in plant disease detection. Models trained only on controlled laboratory images often perform well on similar data but fail to generalize to real field conditions, so TARAS includes both clean and natural images.

The training set combines PlantVillage images with real-world images from PlantDoc and curated external sources. This helps the model learn clear disease patterns from controlled images while also becoming robust to complex backgrounds, lighting changes, and field variability. Source balance is handled through sampling and training strategies rather than strict dataset separation.

Two test sets are used for evaluation. test_pd contains only real-world images from PlantDoc and curated external sources, and it is the primary benchmark for deployment performance. test_balanced combines PlantVillage and real-world images, with a higher proportion of PlantVillage samples, and is used to examine class-level behavior under a more balanced distribution.

This setup exposes the model to real-world variability during training, prioritizes realistic evaluation, and still provides a secondary benchmark for controlled comparison.

2.3.2 Dataset Construction Strategy

The dataset combines laboratory and real-world image sources to address domain shift in plant disease detection. Models trained only on controlled laboratory images often perform well on similar data but fail to generalize to field conditions, so TARAS includes both clean and natural images.

The training set combines PlantVillage images with real-world images from PlantDoc and curated external sources. This helps the model learn clear disease patterns from controlled images while also becoming robust to complex backgrounds, lighting changes, and field variability. Source balance is handled through sampling and training strategies rather than strict separation.

For evaluation, two test sets are used. test_pd contains only real-world images from PlantDoc and curated external sources, and serves as the primary benchmark for deployment performance. test_balanced combines PlantVillage and real-world images, with a higher

proportion of PlantVillage samples, and is used to analyze class-level behavior under a more balanced distribution.

This setup exposes the model to real-world variability during training, prioritizes realistic evaluation, and still provides a secondary benchmark for controlled comparison.

2.3.3 Data Cleaning and Validation

Before training, the dataset is cleaned and validated to ensure consistency across all sources. Since the images come from public datasets and manually collected external sources, removing data quality issues is necessary for reliable training.

First, exact duplicates and near-identical images are removed to prevent overfitting to repeated samples. This is especially important when combining multiple datasets, since the same PlantVillage or external images may appear in more than one repository.

Label consistency is also checked carefully. Real-world images have a higher risk of incorrect or ambiguous labels, so each image is reviewed against its visible symptoms. Images with uncertain or low-confidence labels are excluded.

These steps ensure that training is based on clean, reliable, and representative data, improving the stability and generalizability of the model.

2.3.4 Data Preprocessing Pipeline

A consistent preprocessing pipeline is applied to all images to ensure model compatibility and stable training. Since the dataset includes images from different sources with varying resolutions, formats, lighting conditions, and color profiles, preprocessing standardizes the input data.

All images are resized to the fixed resolution required by the model, while preserving aspect ratio as much as possible to avoid distorting disease patterns. They are then converted into tensors and normalized using standard mean and standard deviation values, aligning the input distribution with the expectations of pre-trained backbone models.

Color channels are also standardized to reduce unnecessary variation between image sources. The same preprocessing steps are used during both training and inference, ensuring that the model receives inputs with consistent statistical properties in both phases.

2.3.5 Data Augmentation Strategy

Data augmentation is used during training to improve generalization to real-world conditions. Since field images vary in orientation, lighting, and background complexity, augmentation helps the model become less dependent on specific visual patterns in the training data.

The pipeline includes random horizontal flipping, small rotations, controlled cropping, and mild brightness and contrast changes. These transformations help the model handle differences in viewpoint, leaf orientation, and illumination.

Augmentations are kept realistic so that disease symptoms are not distorted. They are applied only to the training set, while validation and test sets remain unchanged to ensure reliable and comparable evaluation results.

2.3.6 Dataset Splitting Strategy

The dataset is split into training, validation, and test sets to support both effective learning and reliable evaluation. Since the goal is real-world deployment, validation and test sets are designed to reflect practical usage conditions.

The training set contains 5,913 images across 12 classes, combining PlantVillage images with real-world images from PlantDoc and curated external sources. This hybrid structure helps the model learn clear disease patterns from controlled images while also adapting to more complex field conditions.

Two validation sets are used during training. val_pd contains 182 real-world images and serves as the main validation source for realistic conditions. val_pv contains 518 PlantVillage-like images and provides a more stable reference for model behavior. Model selection is based on a weighted validation score:

$$0.65 \times \text{val_pd Macro F1} + 0.35 \times \text{val_pv Macro F1}$$

This weighting prioritizes real-world performance while still benefiting from the consistency of cleaner data.

For testing, two distinct datasets are defined. The *test_pd* set includes 244 images composed entirely of real-world data collected from PlantDoc and manually curated external sources. This dataset serves as the primary benchmark for evaluating deployment readiness. In contrast,

the *test_balanced* set contains 275 images, consisting of a mixture of PlantVillage and real-world samples, with a higher proportion of PlantVillage images. This dataset is used to analyze class-level performance under a more balanced and controlled distribution.

This splitting strategy ensures that the model is evaluated primarily on realistic conditions while still allowing for controlled comparison and detailed performance analysis across different data distributions.

2.4. Model Architectures

2.4.1 CNN-Based Models

CNN architectures are included as strong baselines for image classification. They are widely used in plant disease detection because they capture local visual features such as texture, color changes, and lesion patterns, which are important for distinguishing diseases.

This study evaluates two CNN-based architectures: EfficientNetV2 and ConvNeXt. EfficientNetV2 offers a strong balance between accuracy and computational efficiency through optimized scaling of depth, width, and resolution. ConvNeXt is a modern CNN that adopts some transformer-inspired design choices while keeping the efficiency of convolutional operations, making it effective for complex visual patterns.

All models are trained and evaluated under the same dataset and experimental conditions. This ensures a fair comparison and helps determine whether transformer-based architectures provide meaningful improvements over strong CNN baselines.

2.4.2 Transformer-Based Models

In addition to CNN-based architectures, transformer-based models are evaluated to assess their effectiveness in plant disease classification under real-world conditions. Unlike traditional CNNs, which primarily focus on local spatial features, transformer models are designed to capture both local and global relationships within an image through self-attention mechanisms. This allows them to model broader contextual information, which can be particularly useful for distinguishing visually similar disease patterns.

In this study, two transformer-based architectures are considered: Swin-Tiny and Swin-Small. Both models are based on the Swin

Transformer architecture, which applies a hierarchical design with shifted window-based self-attention. This structure enables efficient computation while preserving the ability to integrate information across different regions of the image. As a result, Swin models can effectively balance local feature extraction with global context awareness.

The Swin-Small model, in particular, is included as a higher-capacity variant to evaluate whether increased model complexity leads to improved performance on the revised dataset. All transformer-based models are trained under the same experimental conditions as the CNN-based models to ensure a fair and consistent comparison.

By including transformer architectures in the evaluation, the study aims to determine whether their ability to capture long-range dependencies provides an advantage over CNN-based approaches in handling real-world variability and fine-grained disease distinctions.

2.4.3 Architectural Differences

The evaluated architectures differ mainly in how they capture visual information. CNN-based models such as EfficientNetV2 and ConvNeXt use convolutional operations to extract local features like texture, edges, color changes, spots, and lesions. These features are especially important for identifying fine-grained disease symptoms.

Transformer-based models such as Swin-Tiny and Swin-Small use self-attention to model relationships between different image regions. This helps them capture both local details and broader leaf-level context, which is useful when symptoms are subtle or spread across larger areas.

CNNs build spatial hierarchies through stacked convolutional layers, while Swin Transformers use a hierarchical structure with shifted window attention to share information between neighboring regions efficiently. Comparing both model families under the same conditions helps determine which approach is more suitable for plant disease detection.

2.5. Training Configuration

2.5.1 Optimization Strategy

The training process is treated as a multi-class classification problem and optimized using weighted cross-entropy loss. Cross-entropy is suitable for

classification because it compares the predicted class probabilities with the true labels.

Because the dataset is imbalanced, class weights are computed using inverse class frequency. This gives higher importance to underrepresented classes and reduces bias toward majority classes.

Model parameters are updated with a gradient-based optimizer. Using the same weighted loss and optimization settings across all models keeps training fair and makes architecture comparisons more reliable.

2.5.2 Sampling Strategy

Class imbalance is handled with both weighted loss and weighted sampling. A WeightedRandomSampler is used during training so that underrepresented classes are sampled more often and overrepresented classes less often.

Without this, training could be dominated by majority classes, producing high overall accuracy but weak minority-class performance. Weighted sampling balances the examples seen during training, while class-weighted loss gives higher penalty to mistakes on minority classes.

Together, these methods help the model learn more consistent representations across all disease categories.

2.5.3 Hyperparameters

All models are trained with the same hyperparameter settings to ensure a fair comparison. Input resolution, batch size, learning rate, number of epochs, preprocessing, and evaluation metrics are kept consistent across architectures.

Images are resized to a fixed resolution compatible with the models, and training is run for enough epochs to allow convergence. The learning rate is kept stable, while batch size is selected according to hardware limits and efficient resource use.

Instead of tuning each model separately, a unified configuration is used for all experiments. This prevents comparison bias and ensures that performance differences mainly reflect architectural differences rather than model-specific hyperparameter optimization.

2.6. Evaluation Metrics

2.6.1 Classification Metrics

The performance of the disease detection module is evaluated using standard classification metrics that capture both overall accuracy and class-level behavior. Given the multi-class nature of the problem and the presence of class imbalance, multiple complementary metrics are used to provide a comprehensive evaluation.

The primary metrics used in this study are:

- Accuracy
- Precision
- Recall
- Macro F1-score
- Weighted F1-score

These metrics are computed on both validation and test datasets, with particular emphasis on the *test_pd* set, which reflects real-world performance.

Dataset	Purpose	Number of Samples	Description
val_pd	Validation	182	Real-world images
val_pv	Validation	518	PlantVillage images
test_pd	Test	244	Real-world only
test_balanced	Test	275	Mixed PlanVillage, Plantdoc and External Real world Samples

Key Evaluation Focus:

Among these metrics, Macro F1-score is treated as the primary evaluation metric, since it assigns equal importance to all classes regardless of their frequency. This is particularly important in this task, where some disease classes have fewer samples and would otherwise be underrepresented in accuracy-based evaluation.

Accuracy and Weighted F1 are still reported to provide additional context, but they are considered secondary indicators compared to Macro F1 when making model selection decisions.

2.6.2 Metric Justification

Evaluation metrics are selected based on the dataset structure and task requirements. Since plant disease classes are not perfectly balanced, accuracy alone can be misleading; a model may perform well on majority classes while failing on minority ones.

For this reason, Macro F1-score is used as the primary metric. It gives equal importance to all classes by averaging the F1-score of each category, making minority-class performance visible.

Precision and recall are also reported to analyze false positives and false negatives. Accuracy and Weighted F1 are included as complementary metrics to show overall performance and account for class distribution.

Together, these metrics provide a balanced evaluation of both general model performance and class-level behavior.

2.6.3 Validation Selection Metric

Model selection during training is based on a weighted combination of two validation sets in order to balance real-world performance with stability. Since the primary goal of the system is reliable operation under field conditions, greater importance is assigned to the validation set that reflects real-world data.

The final validation score is computed as:

$$0.65 \times \text{val_pd Macro F1} + 0.35 \times \text{val_pv Macro F1}$$

In this formulation, *val_pd* represents the real-world validation set, while *val_pv* corresponds to the PlantVillage-like validation set. By assigning a higher weight to *val_pd*, the selection process prioritizes models that perform well on realistic images. At the same time, the inclusion of *val_pv* helps stabilize evaluation by incorporating cleaner and more consistent data.

This weighted validation strategy ensures that model selection is aligned with deployment requirements while avoiding over-reliance on a single data distribution. As a result, the chosen model demonstrates both

robustness in real-world conditions and consistency across different types of input data.

2.7. Experimental Results

2.7.1 Benchmark Comparison Results

To evaluate the relative performance of different model architectures, a benchmark comparison is conducted under a controlled and consistent experimental setup. All candidate models are trained using the same dataset, preprocessing pipeline, training configuration, and evaluation metrics, ensuring that the comparison is fair and reliable.

The benchmark includes both CNN-based and transformer-based architectures. Their performance is evaluated on the *test_pd* dataset, which serves as the primary real-world benchmark, as well as on the *test_balanced* dataset for additional analysis.

Benchmark Results on Test Sets :

Model	Test_PD Accuracy	Test_PD Macro F1	Test_Balanced Macro F1
ConvNeXt-Small	0.9016	0.8953	0.9151
ConvNeXt-Tiny	0.9057	0.8934	0.9158
EfficientNetV2-rm-M	0.8811	0.8783	0.9011
Swin-Tiny	0.8730	0.8533	0.8848
Swin-Small	**0.9344**	**0.9269**	**0.9412**

The results show strong performance across both evaluation sets, suggesting that the dataset design and training strategy are effective. However, real-world results reveal clear differences between models.

Swin-Small achieves the best performance on test_pd, with the highest accuracy and Macro F1-score. This indicates stronger generalization to real-world images compared to the other architectures. CNN-based models such as ConvNeXt-Small also perform well, but remain slightly below Swin-Small.

The difference is especially important for Macro F1-score, since it reflects balanced performance across all classes. Swin-Small therefore appears more consistent across both frequent and less frequent disease categories.

Overall, the benchmark results support selecting Swin-Small as the final deployment model.

2.7.2 Final Model Results

Dataset	Accuracy	Macro F1	Weighted F1
test_pd	0.9344	0.9269	0.9338
test_balanced	0.9418	0.9412	0.9415

The selected model performs strongly on the real-world test_pd set, which is the primary benchmark for deployment. Its high Macro F1-score shows balanced performance across classes, not only strong results on frequent categories.

Performance on test_balanced is slightly higher, which is expected because it has a more controlled distribution and cleaner samples. However, the small gap between test_pd and test_balanced suggests that the model generalizes well and does not depend on easier data.

The high Weighted F1-score also confirms stable overall performance across both frequent and less frequent classes. These results show that the selected model is accurate, robust, and suitable for deployment in TARAS.

2.7.3 Per-Class Performance Analysis

In addition to overall performance metrics, the behavior of the model is analyzed at the class level to better understand how it performs across different disease categories. This analysis provides insight into which classes are learned effectively and which remain more challenging.

Test_PD_Results:

Classification Report:

Precision	Recall	f1-score	Support
Bell_pepper_bacterial_spot	0.8000	0.8889	0.8421
Bell_pepper_healthy	0.9000	0.8182	0.8571
Corn_common_rust	1.0000	0.9000	0.9474
Corn_northern_leaf_blight	0.9167	1.0000	0.9565
Potato_Leaf_blight	0.9167	0.7333	0.8148
Squash_powdery_mildew	1.0000	1.0000	1.0000
Tomato_Late_blight	0.9310	1.0000	0.9643
Tomato_Leaf_Mold	1.0000	0.9688	0.9841
Tomato_Leaf_blight	0.8649	0.9143	0.8889
Tomato_Septoria_leaf_spot	0.9333	0.9032	0.9180
Tomato_Spider_mites_Two_spotted_spider_mite	1.0000	1.0000	1.0000
Tomato_healthy	0.9333	0.9655	0.9492

Test_Balanced_Results:

Classification Report:

Precision	Recall	f1-score	Support
Bell_pepper_bacterial_spot	0.8667	0.9286	0.8966
Bell_pepper_healthy	0.9286	0.8667	0.8966
Corn_common_rust	1.0000	0.9412	0.9412
Corn_northern_leaf_blight	0.9412	1.0000	0.9697
Potato_Leaf_blight	0.9375	0.7895	0.8571
Squash_powdery_mildew	1.0000	1.0000	1.0000
Tomato_Late_blight	0.9310	1.0000	0.9643
Tomato_Leaf_Mold	1.0000	0.9688	0.9841
Tomato_Leaf_blight	0.8649	0.9143	0.8889
Tomato_Septoria_leaf_spot	0.9333	0.9032	0.9180
Tomato_Spider_mites_Two_spotted_spider_mite	1.0000	1.0000	1.0000
Tomato_healthy	0.9333	0.9655	0.9492

Per-class results are analyzed to understand how the model performs across different disease categories. The classification reports for test_pd and test_balanced are presented in the tables above.

Across both datasets, the model performs strongly for most classes. Categories such as Squash_powdery_mildew and Tomato_Spider_mites_Two_spotted_spider_mite achieve perfect or near-perfect scores, suggesting that their visual symptoms are distinctive and easy for the model to learn.

Some classes remain more challenging. Potato_Leaf_blight has lower recall and F1-scores in both datasets, likely because it visually overlaps with other blight-related diseases. Smaller variations are also seen in classes such as Bell_pepper_bacterial_spot and Bell_pepper_healthy, showing that healthy and diseased leaves can sometimes be difficult to separate.

Performance on test_balanced is generally slightly higher and more stable, which is expected because it contains a more controlled

distribution and cleaner samples. In contrast, test_pd shows more variation due to the difficulty of real-world images.

Overall, the per-class analysis shows strong performance across most categories while identifying a few classes that need further improvement.

2.7.4 Confusion Matrix Analysis

To further analyze model errors, confusion matrices for both *test_pd* and *test_balanced* datasets are examined. The matrices illustrate the relationship between true class labels and predicted outputs, allowing detailed inspection of misclassification patterns.

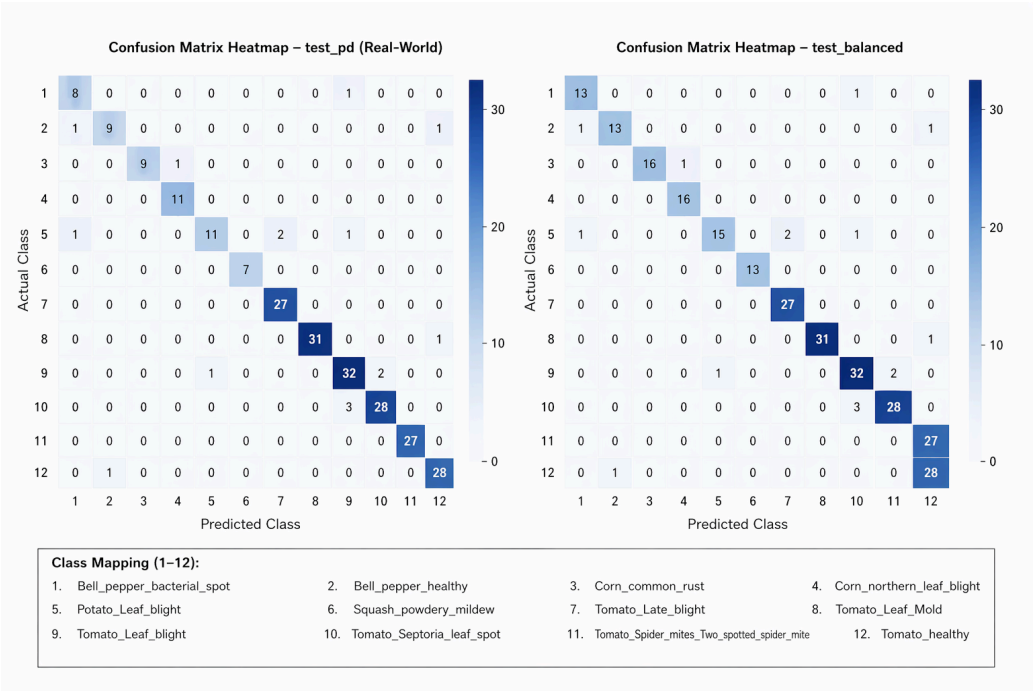

Figure 1: Confusion matrices for *test_pd* (real-world) and *test_balanced* datasets.

The confusion matrices show strong diagonal dominance in both datasets, meaning most samples are correctly classified. This matches the high accuracy and Macro F1-scores reported earlier.

Most errors occur between visually similar disease classes. Potato_Leaf_blight is confused with other blight-related categories, which explains its lower recall in the per-class results. Similar minor confusion appears between Tomato_Leaf_blight and Tomato_Septoria_leaf_spot, likely due to shared symptoms such as spot patterns and lesion structure.

Some healthy and diseased classes are also occasionally confused, especially when symptoms are weak or unclear. This is more visible in test_pd, where real-world lighting, background, and image quality make classification harder.

Compared with test_pd, test_balanced shows cleaner class separation and fewer off-diagonal errors, as expected from its more controlled distribution. Overall, the errors are concentrated in a few visually similar classes rather than spread randomly, suggesting that the model has learned meaningful disease features and that remaining mistakes are mostly due to visual ambiguity.

2.8. Generalization and Robustness Analysis

2.8.1 Real-World Generalization

The model's ability to generalize to real-world conditions is evaluated by comparing test_pd, which contains only real-world images, with test_balanced.

The results show only a small performance drop on test_pd, meaning the model does not depend heavily on clean or controlled data. Instead, it learns robust features that transfer well to more complex field images.

Despite challenges such as lighting variation, background clutter, and partial occlusion, the model remains consistent across most classes. This supports the effectiveness of the hybrid training strategy that combines laboratory and real-world data.

Overall, the model is suitable for practical use in TARAS and can operate reliably under real agricultural conditions.

2.8.2 Class Stability Analysis

Class stability measures whether the model maintains similar performance across classes and datasets. This is important because a model with high overall accuracy may still be unreliable if some classes perform poorly.

Most classes show stable results across both test_pd and test_balanced. Categories such as Squash_powdery_mildew, Tomato_Spider_mites_Two_spotted_spider_mite, and Tomato_Leaf_Mold

consistently achieve high precision, recall, and F1-scores, showing that the model learned reliable features for these diseases.

Some classes vary more. Potato_Leaf_blight has lower recall and F1-scores in both datasets, especially under real-world conditions. Minor fluctuations also appear in Bell_pepper_bacterial_spot and Bell_pepper_healthy, where visual differences can become less clear.

Overall, the model does not show major class-specific performance drops. This suggests that it performs not only accurately, but also consistently across different data distributions, which is important for deployment in TARAS.

2.8.3 Failure Case Analysis

Failure cases are analyzed to understand the model's limitations beyond overall performance metrics. Although the model performs strongly, incorrect predictions reveal where the task remains difficult.

Most errors come from visual similarity between disease classes. Potato_Leaf_blight, Tomato_Late_blight, and Tomato_Leaf_blight can share symptoms such as dark spots, irregular discoloration, and similar lesion patterns. When these differences are subtle, the model may confuse them.

Another source of error is early-stage or mild disease, where symptoms are weak or ambiguous. These cases may resemble healthy leaves or overlap with multiple disease categories, especially in real-world images with poor lighting, shadows, occlusion, or cluttered backgrounds.

Some failures are also linked to data limitations, such as fewer samples for certain classes or missing variations in disease appearance. However, the number of errors is relatively small and concentrated in a few challenging categories, suggesting that most mistakes come from visual ambiguity rather than a fundamental weakness in the model.

3. LLM Module Evaluation

3.1 Evaluation Methodology

Evaluation is qualitative.

3.2 Scenario-Based Testing

Various real-world scenarios are tested.

3.3 Tool Usage Analysis

Tool-based reasoning is analyzed.

3.4 Latency Analysis

Response times are measured.

3.5 Reliability and Grounding

The system ensures grounded responses.

3.6 Failure Cases

Limitations are identified.